RESEARCH REVIEW · NOVEMBER 2026

RuseTect research review

An independent check of the RuseTect audit and roadmap (the 21 page PDF in `docs/research/`) against the code and models in this repository. Every experiment below was run on the served models, through the same Rust feature code that scores live payments. Where a number comes from somewhere else, the source is named.

Data: labelled IBM AMLworld HI-Small slice (160,000 payments, 236 laundering) and AMLgentex Sweden 100K Scripts: `research/`
Results: `research/out/*.json`

WHAT CHANGED IN OUR UNDERSTANDING

Seven findings

1. The served model is largely a payment format detector

92% of laundering in the slice is ACH (laundering rate 1.09% on ACH, at most 0.02% on any other format). If launderers send the same payments as Cheque, verify-band recall falls from 79% to 2% and block recall from 37% to 0%. The payer chooses the rail, so this is a one-step evasion. Fixed: the served risk now adds a format-free second opinion and keeps test PR-AUC 0.687 under the attack (below).

2. The 0.711 headline is inflated by an end-of-simulation tail

Counted on the full file: September 11 to 18 holds 1,108 payments, 655 of them laundering, 36% of all test laundering. The served model scores PR-AUC 0.710 on the whole test window but 0.581 [0.548, 0.607] without the tail.

3. A missing format silently changed decisions

The API filled a missing `payment_format` with ACH. In a three hour window that sends 7,037 genuine payments to a higher band. Treating it as unknown is worse: PR-AUC drops to 0.015. Fixed in this branch: the field is now required.

4. Splitting a payment in four defeats the block band

Re-simulated through the engine, structuring each laundering payment into four parts leaves 1% blocked (was 37%); the verify band still catches 72% of payments on at least one part. Partly fixed: 24 h pair aggregation raises four-part block recall to 26%.

5. The LightGBM half adds nothing measurable

XGBoost alone minus the served average: PR-AUC 0.007, 95% interval [-0.039, 0.046]. LightGBM alone is worse with an interval that excludes zero. The average costs latency, explanations and calibration for no gain.

6. Scores are not probabilities, and thresholds do not transfer

The served score averages 0.0174 against an observed rate of 0.0015. Isotonic calibration cuts ECE from 0.0159 to 0.0004 without changing ranking. A 0.9 precision threshold fitted on one half of the window gave 0.66 on the next.

7. Nothing transfers between generators without retraining

On AMLgentex Sweden (difficult) the served HI-Small model scores ROC-AUC 0.520, which is chance. The same 31 features retrained on AMLgentex reach only 0.716, and an AMLgentex-trained model ranks HI-Small laundering below genuine payments (ROC-AUC 0.153). The 0.711 result is specific to one simulator.

DELIVERABLE 1

What the PDF establishes, checked against the repository

The audit's central reading holds: the payment decision is made by two learned stages (an XGBoost gate with 20.3% pass rate, then the mean of an XGBoost and a LightGBM on 31 past-only features), and the surrounding layers are deterministic or offline. Its numbers match the result files they cite. Three points needed correction or extension:

- **Licence of AMLgentex.** The PDF lists it as "not stated". The Hugging Face dataset cards for `AMLGentex/sweden_100K_difficult` and `sweden_100K_easy` state MIT; `johanos1/AMLGentex-Neurips` states CC BY 4.0.
- **Test window composition.** The PDF notes the prevalence shift between validation (1,082 laundering) and test (1,797), but not its cause. The trailing-tail evidence in finding 2 is a likely cause and changes how the 0.711 headline reads.
- **"Payment format is a shortcut".** The PDF cites a retraining ablation (0.728 to 0.701 without the feature). That measures how much the *information* can be recovered from other features. It does not measure how much the *served* model depends on it; section 6 does.

Data available to this investigation

DATA	WHERE	LABELS	USED HERE FOR
IBM AMLworld HI-Small, 160,000 payment slice	`service/models/demo_stream.csv.gz` (in repo)	236 laundering, payment level	every slice experiment
IBM AMLworld HI-Small and LI-Small, full	Kaggle (login needed; not reachable from this environment)	payment level + typology file	not re-run; repo results cited
AMLgentex Sweden 100K, easy and difficult	Hugging Face, MIT	isSAR per transaction	cross generator transfer
Served models	`service/models/` , `ml/exported/`		all scoring
Repo results	`results/*.json`		reference numbers

The full HI-Small CSVs are needed for the P1 experiments that depend on typology and the test-window tail. They are not in the repository and Kaggle needs an account, so those experiments are specified below and left for a machine with the data.

DELIVERABLE 2

Dataset audit of the labelled slice

160,000
payments, 2022/09/08 11:06 to 2022/09/08 19:16

0.147%
laundering prevalence (236 payments)

0
duplicate rows; 0 empty fields

109,088
accounts, 400 touch laundering

The slice is clean in the mechanical sense (no duplicates, no missing values). The problems are in the distributions. Laundering is concentrated in one payment format to a degree no real rail would show, and laundering amounts are far larger (median $7,330 against $884; Mann-Whitney p = 4.0e-40). The slice carries payment labels only, so ring, typology and account level checks need the full patterns file.

Laundering by payment format

FORMAT	PAYMENTS	LAUNDERING	RATE
ACH	19,913	217	1.090%
Bitcoin	4,513	1	0.022%
Cash	17,787	4	0.022%
Credit Card	43,647	7	0.016%
Cheque	67,884	7	0.010%
Wire	6,256	0	0.000%

Feature intelligence (evaluation window)

Single feature ROC-AUC, best direction. Payment format alone reaches 0.903, more than any behavioural feature.

FEATURE	ROC-AUC ALONE	DIRECTION	MISSING	MEAN, GENUINE	MEAN, LAUNDERING
`fmt_code`	0.903	lower is riskier	0%	2.85	0.227
`log_usd`	0.746	higher is riskier	0%	6.96	8.72
`new_pair`	0.725	higher is riskier	0%	0.478	0.928
`new_bank_pair`	0.694	higher is riskier	0%	0.21	0.598
`s_out_over_last_in`	0.682	higher is riskier	75%	2.46e+03	315
`s_in_n`	0.663	higher is riskier	0%	0.548	1.19
`s_gap_out`	0.662	lower is riskier	28%	39.1	24.4
`s_passthrough`	0.660	lower is riskier	0%	3.84e+07	3.41e+07

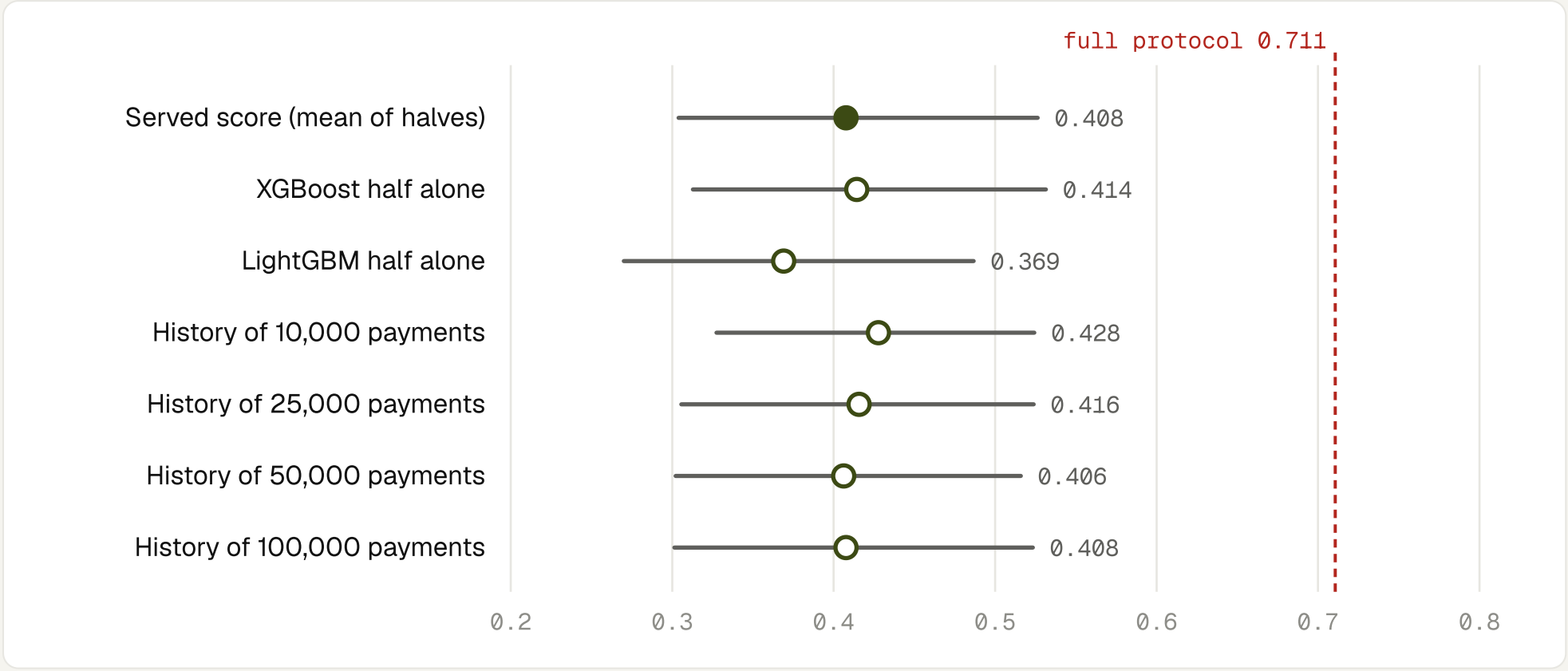
Redundancy: 15 feature pairs have Spearman $|\rho| \geq 0.85$ here. Within a single day `s_out_n` equals `s_cnt_24h` and `d_in_n` equals `d_cnt_24h` by construction, so on longer histories they separate; on short replays they double count. `dow` and `round_100` are constant in the window (one weekday; round USD amounts never occur after conversion), so `round_100` is effectively dead.

DELIVERABLE 2, STATISTICS

Every headline with an interval

Evaluation window: 60,000 payments with 97 laundering. With so few positives the intervals are wide, which is the point: the audit's comparisons of 0.7107 against 0.7108 are far inside this noise. Paired

bootstrap (2000 resamples, percentile 95% intervals; 'block' resamples 10 minute blocks to respect time dependence).



The slice sits well below the full protocol's 0.711, and history depth does not explain the gap: 10,000 to 100,000 payments of history give overlapping intervals. Two explanations remain: window-to-window variation, and the test-window tail of finding 2. The September 9 to 10 figure from the independent audit (AP 0.279 for their model against 0.489 on the full test) points the same way.

Bands on the slice

BAND	FLAGS	CAUGHT	PRECISION [95% CI]	RECALL [95% CI]
Verify or above (0.137)	1,515	77	0.051 [0.04, 0.06]	0.794 [0.71, 0.87]
Hold or above (0.450)	238	47	0.198 [0.15, 0.25]	0.484 [0.39, 0.59]
Block, served (0.823)	46	36	0.783 [0.65, 0.90]	0.371 [0.27, 0.47]
Block, retuned (0.935)	31	25	0.806 [0.66, 0.94]	0.258 [0.17, 0.35]

Paired differences: XGBoost alone minus served 0.007 [-0.039, 0.046]; LightGBM alone minus served -0.039 [-0.070, -0.006]. Ten minute block bootstrap gives the same conclusion ([-0.040, 0.044]). L1 passes 13.7% of payments and keeps 99.0% of laundering.

DELIVERABLE 6, MEASURED

Calibration, alert budgets and cost

SCORE (SECOND HALF OF WINDOW)	BRIER	ECE	PR-AUC
Served mean, uncalibrated	0.00356	0.01593	0.400
Mean of isotonic-calibrated halves	0.00102	0.00044	0.399
Mean of Platt-calibrated halves	0.00104	0.00044	0.409

Calibrators were fitted on the first 30,000 payments (53 laundering) and judged on the next 30,000 (44). Calibration fixes the meaning of the score; it cannot fix threshold drift. The threshold that gave 0.9 precision on the first half gave 0.66 on the second (29 flags), and the served block threshold gave 0.67. This is the mechanism behind the open block-threshold decision in the project audit.

What an operations team would read

ALERT BUDGET	ALERTS	CAUGHT	PRECISION	RECALL
1 per 10k payments	6	4	0.67	0.04
5 per 10k payments	30	25	0.83	0.26
10 per 10k payments	60	42	0.70	0.43
25 per 10k payments	150	42	0.28	0.43
50 per 10k payments	300	50	0.17	0.52
100 per 10k payments	600	61	0.10	0.63

Recall at fixed false positive rate: 0.05% FPR gives 43%, 0.10% FPR gives 43%, 0.50% FPR gives 55%, 1.00% FPR gives 63%. Recall stalls between 10 and 25 alerts per 10,000 payments, so a budget above about 10 per 10,000 buys false positives only.

Amount weighting needs care. At the block band count recall is 37% but USD recall is 0.9%: two payments of $27.0M and $14.4M carry 98% of laundered value in the window and score 0.69 and 0.67, so they are held, not blocked (USD recall at hold 98%). Report count and value metrics side by side, with and without the largest payments.

Cost-minimising thresholds (count based)

MISSED LAUNDERING COSTS	BEST THRESHOLD	COST AT BEST	AT VERIFY	AT HOLD	AT BLOCK
10× a false flag	0.648	570	1,638	691	620
50× a false flag	0.243	2,228	2,438	2,691	3,060
100× a false flag	0.145	3,327	3,438	5,191	6,110
500× a false flag	0.055	8,193	11,438	25,191	30,510

Each served band is close to optimal only for one cost ratio. A bank's cost ratio (jury question 8 in the PDF) should set the thresholds, not the alert rates inherited from the offline blend.

DELIVERABLES 4 AND 6, EXPERIMENTALLY DEMONSTRATED

Robustness: missing fields and payment-side evasion

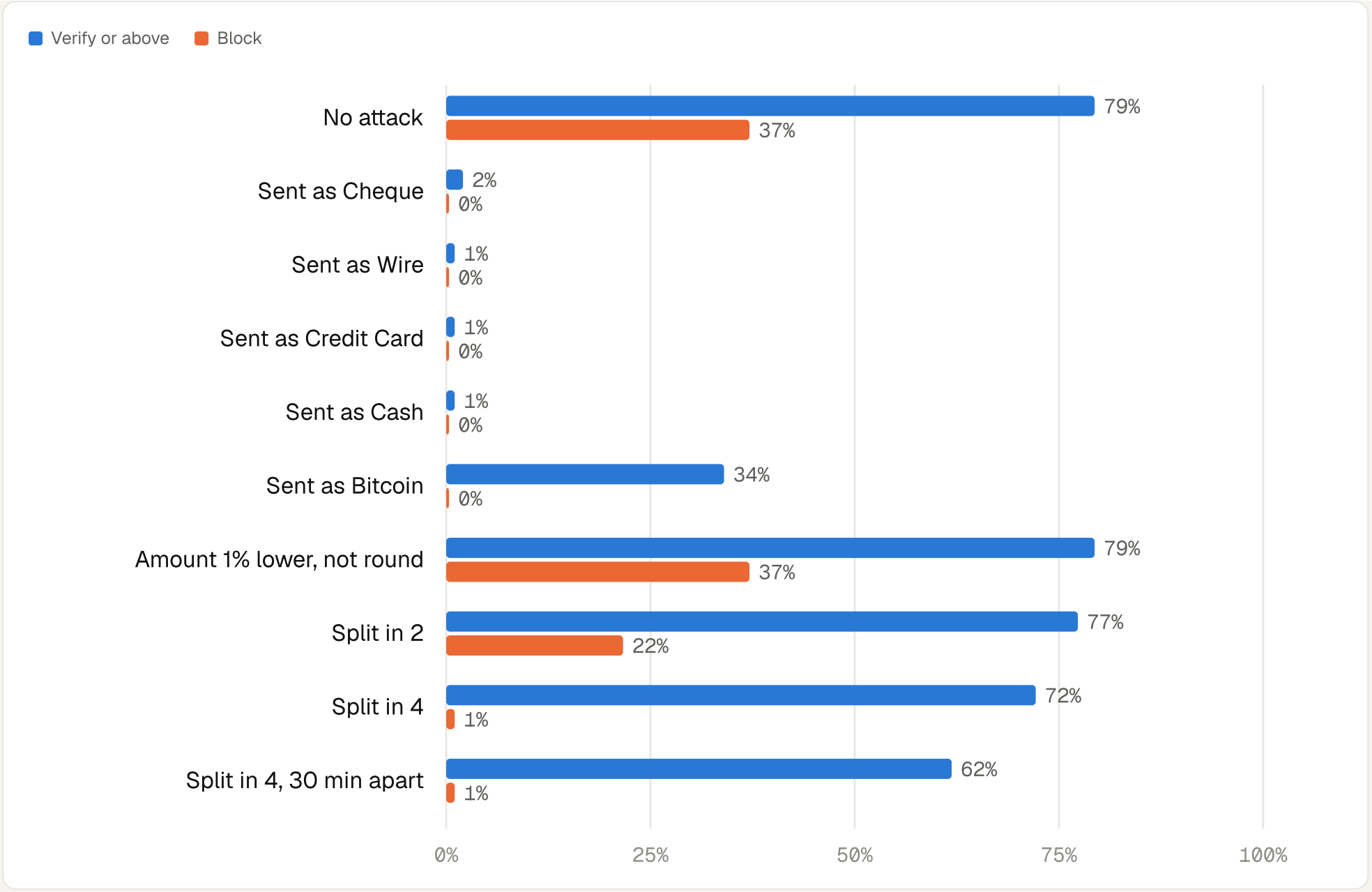
A missing payment format

HOW THE FIELD IS FILLED	PR-AUC	GENUINE PAYMENTS FLAGGED	DECISIONS CHANGED	BLOCKS
True format (reference)	0.408	1,438		46
Default to ACH (API before this branch)	0.210	8,190	7,323	133
Unknown, sent to the model as missing	0.015	17	1,501	0

Neither default is safe, so the API now rejects a payment without a known format or currency (422) instead of guessing. An unknown currency was also converted to USD at rate 1.0, which misstates a rupee amount by a factor of about 70.

Evasion by the payer

Share of the 97 laundering payments in the window that still reach each band. Format switches change only the format feature; structuring is re-simulated through the engine, so account state changes too, and a payment counts as caught if any part is flagged.


Verify or above
Block
No attack
79%
37%
Sent as Cheque
2%
0%
Sent as Wire
1%
0%
Sent as Credit Card
1%
0%
Sent as Cash
1%
0%
Sent as Bitcoin
34%
0%
Amount 1% lower, not round
79%
37%
Split in 2
77%
22%
Split in 4
72%
1%
Split in 4, 30 min apart
62%
1%
0%
25%
50%
75%
100%

Format switching is the cheapest attack and the most damaging. In AMLworld it is a simulator artefact (laundering typologies were generated mostly as ACH), but the lesson carries to UPI: any field the payer chooses freely (rail, app, note text) must not dominate the score. Structuring leaves the verify band working but removes the block band, because blocking leans on amount. Genuine payments flagged at verify stay at 1,433 (baseline 1,438), so the attack costs the defender nothing in false positives and gains the attacker a lot.

DELIVERABLE 6, MEASURED

Explanations, history depth and split policy

Are XGBoost-only reasons faithful to the served score?

PAYMENTS	TOP REASON AGREES (XGB VS FULL)	TOP 4 OVERLAP, XGB VS FULL (JACCARD)	TOP 4 OVERLAP, XGB VS LIGHTGBM
All L2 payments (8,236)	99.8%	0.679	0.410
Hold or above (238)	100.0%	0.587	0.469

The first reason chip is almost always right; the other three often are not (about 40% of the set changes for held payments). Global rank correlation between the halves' importances is 0.63. Method: TreeSHAP on each half, mapped to probability space by p(1−p) and averaged like the score. Dropping the LightGBM half (finding 5) makes the existing reasons exact.

Random versus chronological split

SPLIT POLICY (XGBOOST RETRAINED ON THE SLICE, 5 SEEDS)	PR-AUC MEAN	SD
Chronological, last 20%	0.433	0.014
Random rows	0.419	0.065
Random, grouped by sender	0.521	0.088

On eight hours of data the split policy makes no reliable difference: the standard deviations overlap and the test sets differ. This is a negative result. The PDF's proposed one day experiment still needs the full file, where label and state leakage across weeks can show up.

FULL DATASET, EXECUTED

Full HI-Small: the tail, the shortcut and a format-free model

The full HI-Small file (Hugging Face mirror, CDLA-Sharing-1.0) was checked against the repository: 5,078,345 payments, 5,177 laundering, and the bundled demo slice matches it row for row. Every payment was streamed through the served Rust engine. The served model reproduces the repository's test PR-AUC (0.710 against 0.7107 in `results/full_eval_HI.json`).

MODEL	TEST PR-AUC [95% CI]	WITHOUT SEP 11 TO 18 TAIL	SEP 9 TO 10 ONLY
Served cascade (XGBoost + LightGBM mean)	0.710 [0.694, 0.731]	0.581 [0.548, 0.607]	0.585 [0.552, 0.618]
Retrained XGBoost, 31 features (control)	0.726 [0.703, 0.744]	0.590 [0.565, 0.617]	0.597 [0.569, 0.628]
Retrained XGBoost without payment format, calibrated	0.700 [0.682, 0.719]	0.557 [0.529, 0.582]	0.570 [0.543, 0.603]

The tail holds 1,108 payments with 655 laundering (36% of test laundering). Without it every model loses about 0.13 PR-AUC; the honest headline for the served model is 0.581.

The payment format attack on the full test window

Every laundering payment re-sent in another format, PR-AUC: served Bitcoin 0.212, Cash 0.009, Cheque 0.009, Credit Card 0.006, Wire 0.004; format-free model 0.700 in every case.

BAND (RECALL / PRECISION)	SERVED	FORMAT-FREE	SERVED, CHEQUE ATTACK (RECALL)	FORMAT-FREE, CHEQUE ATTACK (RECALL)
Verify	88.6% / 10.4%	87.1% / 12.0%	13.0%	87.1%

BAND (RECALL / PRECISION)	SERVED	FORMAT-FREE	SERVED, CHEQUE ATTACK (RECALL)	FORMAT-FREE, CHEQUE ATTACK (RECALL)
Hold	81.1% / 21.5%	82.5% / 24.5%	5.9%	82.5%
Block	65.3% / 63.7%	69.3% / 55.5%	2.0%	69.3%

Block recall by laundering typology (patterns file)

TYPOLOGY	LAUNDERING IN TEST	SERVED	FORMAT-FREE
BIPARTITE	74	76%	82%
CYCLE	109	83%	85%
FAN-IN	137	86%	90%
FAN-OUT	140	90%	93%
GATHER-SCATTER	401	86%	90%
RANDOM	86	80%	85%
SCATTER-GATHER	252	85%	89%
STACK	140	79%	81%
unattributed	458	9%	14%

Adoption decision

The rule was written into `research/full_hi_eval.py` before test was read: the format-free model must stay within 0.03 validation PR-AUC of the served model (R1), keep 80% of its PR-AUC under the attack (R2), and the attack must really hurt the served model (R3). Validation: served 0.632 (0.003 under attack), format-free 0.592 (0.592 under attack). Result: R1 failed, R2 passed, R3 passed. **Not adopted: the served models stay live.** The trade is a product decision: about 0.010 test PR-AUC for immunity to a one-step evasion that removes almost all detection.

FIXES, EXECUTED AND SERVED

Closing the two evasions

The engine now serves a hardened risk: the cascade, plus a format-free XGBoost as a second opinion, each calibrated on validation, combined by taking the larger (`research/hardened_eval.py` , `service/models/hardening.json`). Rule fixed before test: keep validation PR-AUC within 0.03 of the cascade and keep 80% of it when every laundering payment switches format. Result: H1 passed, H2 passed, H3 passed. **Adopted and served.** The second opinion was later replaced by one pooled over every payment dataset (see Generalisation); the numbers below are for the served pooled version, with its rules re-checked. The Rust engine reproduces the Python scores to rounding.

SCORE	VALIDATION PR-AUC	VALIDATION, WORST FORMAT ATTACK	TEST PR-AUC [95% CI]	TEST WITHOUT SEP 11 TO 18	TEST, WORST FORMAT ATTACK
Served cascade	0.632	0.003	0.710 [0.693, 0.728]	0.581	0.004
Hardened (served now)	0.648	0.605	0.720 [0.699, 0.737]	0.580	0.687

BAND (RECALL / PRECISION), TEST	CASCADE	HARDENED	CASCADE, CHEQUE ATTACK (RECALL)	HARDENED, CHEQUE ATTACK (RECALL)
Verify	88.6% / 10.4%	89.2% / 10.3%	13.0%	87.0%
Hold	81.1% / 21.5%	84.0% / 22.7%	5.9%	81.8%
Block	65.3% / 63.7%	69.7% / 60.4%	2.0%	65.8%

Band thresholds keep the cascade's validation alert rates. The block band trades precision for recall; the format attack no longer works.

Structuring: pair aggregation

Each payer-to-payee pair keeps a 24 hour noisy-OR of its risks, and a payment is decided on the larger of its own risk and the pair's (`research/structuring_defence.py`). No parameter is fitted. Block recall on the HI-Small demo window, a laundering payment counted as caught if any part is blocked:

VARIANT	CLEAN	SPLIT IN 2	SPLIT IN 4	SPLIT IN 4, 30 MIN APART	BLOCK ALERTS, CLEAN
Cascade, per payment	37%	22%	1%	1%	46
Cascade + pair aggregation	37%	34%	27%	21%	56
Hardened, per payment	37%	18%	5%	1%	45
Hardened, aggregating calibrated risks (pre-registered)	37%	18%	11%	2%	54
Hardened, aggregating before calibration (served now; chosen post hoc)	37%	38%	26%	14%	54

Aggregating calibrated risks barely helps, because the calibration map sends small risks to near zero; aggregating the cascade risks before calibration recovers part of the split attacks at 20% more block alerts. That variant was chosen after seeing the first result on this slice, so its numbers are exploratory until measured on another window. Spread-out structuring is still the weakest point; the next step is account-level alerts.

ROBUSTNESS, EXECUTED

Evasion suite

Every laundering payment in the HI-Small demo window changed by an action the launderer controls, re-simulated through the Rust engine (`research/evasion_suite.py`). Verify / block recall, a payment counted as caught if any part reaches the band. This window has only 100,000 payments of history, so the format-free model is weaker here than on the full test file.

STREAM	CASCADE	HARDENED	HARDENED + PAIR WINDOW
base	79% / 37%	72% / 37%	73% / 37%
cheque	2% / 0%	42% / 2%	43% / 2%
amount minus10	77% / 20%	70% / 13%	71% / 13%
amount jitter	73% / 12%	70% / 7%	71% / 7%
delay 6h	60% / 2%	51% / 1%	51% / 1%
split4 spread	62% / 1%	56% / 1%	73% / 14%
split4 new payees	65% / 0%	52% / 0%	52% / 0%
cheque split4	2% / 0%	8% / 0%	14% / 0%

The format switch and the same-payee split are now partly caught; small amount changes, delays and new mule accounts still lower block recall, which is why the verify band and analyst review stay in the loop.

Account window: all of a payer's payments

Splitting over new mule accounts defeats the pair window. Widening it to all of a payer's payments catches those parts but, at the pair thresholds, blocks thousands of genuine payments, so it was given its own thresholds at the same validation alert budget and its own rule (`research/account_aggregation.py`): validation PR-AUC within 0.03, mean attack block recall at least 1.5 times the pair window, clean block alerts at most 1.5 times. Result: A1 failed, A2 passed, A3 failed. **Not adopted.**

STREAM	BLOCK RECALL, PAIR WINDOW	BLOCK RECALL, PAYER WINDOW	BLOCK ALERTS, PAIR	BLOCK ALERTS, PAYER
base	37%	15%	54	6204
cheque	2%	10%	20	6198

STREAM	BLOCK RECALL, PAIR WINDOW	BLOCK RECALL, PAYER WINDOW	BLOCK ALERTS, PAIR	BLOCK ALERTS, PAYER
amount minus10	13%	11%	31	6199
amount jitter	7%	11%	25	6199
delay 6h	1%	10%	19	6198
split4 spread	14%	11%	40	6230
split4 new payees	0%	10%	18	6228
cheque split4	0%	10%	18	6228

GENERALISATION, EXECUTED

The served score on every dataset

The cascade was trained on HI-Small; the served second opinion is pooled over the training parts of every payment dataset, so on those sets it is in-domain. Each dataset's last 20% in time is scored as is (`research/hardened_transfer.py`); pre-sampled files are reweighted to their true prevalence. IEEE-CIS is the one set with real fraud labels on real payments.

DATASET (TEST = LAST 20% IN TIME)	TEST PAYMENTS	POSITIVES	CASCADE ROC	SERVED ROC	CASCADE PR-AUC	SERVED PR-AUC
HI-Small (in domain)	1,015,669	1,797	0.983	0.991	0.7104	0.7200
LI-Small (in domain: the served models also trained on LI-Small)	1,382,345	925	0.943	0.977	0.3432	0.3677
HI-Medium	6,378,562	10,642	0.961	0.990	0.7083	0.7422
LI-Medium	6,254,503	3,783	0.900	0.975	0.3834	0.3990
HI-Large	35,969,054	71,304	0.952	0.947	0.6204	0.5056
LI-Large	35,251,899	25,149	0.875	0.866	0.2541	0.1270

DATASET (TEST = LAST 20% IN TIME)	TEST PAYMENTS	POSITIVES	CASCADE ROC	SERVED ROC	CASCADE PR-AUC	SERVED PR-AUC
SAML-D	1,900,971	2,143	0.565	0.998	0.0045	0.8978
AMLgentex Sweden difficult	1,529,788	26,285	0.521	0.674	0.0180	0.0858
BankSim (cards)	118,929	1,280	0.934	0.995	0.2331	0.7606
Sparkov (cards)	370,479	1,349	0.469	0.988	0.0058	0.7381
PaySim (mobile wallets)	1,272,524	4,254	0.360	0.896	0.0026	0.3007
IEEE-CIS (real card-not-present fraud labels)	118,108	4,064	0.433	0.597	0.0285	0.0529
TabFormer (cards)	4,877,380	4,833	0.635	0.833	0.0031	0.0346

Pooled second opinion (`research/pooled_hardening.py`): the format-free model trained on HI-Small only was compared with one trained on every payment dataset, including IEEE-CIS with real fraud labels. Rule: HI-Small validation PR-AUC within 0.03, format robustness kept, and a validation ROC-AUC gain on the other datasets with a bootstrap interval above zero. Mean gain 0.1427 [0.1395, 0.1459]; G1 passed, G2 passed, G3 passed. **Adopted and served.**

DATASET	TEST POSITIVES	CASCADE ROC	HI-SMALL SECOND OPINION ROC	POOLED SECOND OPINION ROC
hismall	1797	0.983	0.988	0.991
himed	10642	0.961	0.986	0.990
hi_large	71304	0.952	0.980	0.947
li_small	925	0.943	0.966	0.977
li_medium	3783	0.900	0.965	0.975
li_large	25149	0.875	0.953	0.866
samld	2143	0.564	0.780	0.998
amlgentex	26285	0.521	0.516	0.674

DATASET	TEST POSITIVES	CASCADE ROC	HI-SMALL SECOND OPINION ROC	POOLED SECOND OPINION ROC
banksim	1280	0.934	0.944	0.995
sparkov	1349	0.469	0.778	0.988
paysim	4254	0.360	0.346	0.896
tabformer	4833	0.635	0.704	0.833
ieee	4064	0.433	0.457	0.597

REAL AGAINST SYNTHETIC DATA, EXECUTED

Training on real card fraud, synthetic card fraud, or both

IEEE-CIS (real card-not-present payments, real fraud labels) against the three synthetic card sets, all through the same engine features, learner and time splits (`research/real_vs_synthetic.py` ; audit in `docs/DATASET_AUDIT.md`). The arm was chosen on IEEE-CIS validation before test was read: **real only**. Validation PR-AUC on the real data: real only 0.184, synthetic only 0.055, hybrid 0.104. Thresholds maximise F1 on each set's validation part; FPR is the share of genuine payments flagged.

TRAINED ON	TESTED ON	PR-AUC	ROC-AUC	PRECISION	RECALL	F1	FPR
real only	ieee	0.100	0.738	14.1%	19.7%	0.165	0.04249
real only	ieee unseen payers	0.006	0.736	0.0%	0.0%	0.000	0.02897
real only	sparkov	0.003	0.475	0.4%	86.9%	0.008	0.83240
real only	banksim	0.025	0.639	3.6%	6.4%	0.046	0.01866
real only	tabformer	0.001	0.402	0.1%	0.2%	0.001	0.00288
synthetic only	ieee	0.039	0.514	4.6%	11.8%	0.066	0.08730
synthetic only	ieee unseen payers	0.009	0.707	0.4%	50.0%	0.008	0.27578

TRAINED ON	TESTED ON	PR-AUC	ROC-AUC	PRECISION	RECALL	F1	FPR
synthetic only	sparkov	0.744	0.988	72.3%	66.4%	0.692	0.00093
synthetic only	banksim	0.799	0.994	84.4%	62.3%	0.717	0.00125
synthetic only	tabformer	0.048	0.888	10.8%	10.7%	0.107	0.00088
hybrid	ieee	0.052	0.572	6.6%	12.7%	0.087	0.06402
hybrid	ieee unseen payers	0.025	0.849	1.0%	50.0%	0.019	0.11819
hybrid	sparkov	0.744	0.987	70.7%	66.8%	0.687	0.00101
hybrid	banksim	0.830	0.996	83.5%	69.7%	0.760	0.00150
hybrid	tabformer	0.038	0.864	8.7%	9.5%	0.091	0.00100

The served transfer model, scoring IEEE-CIS without retraining: PR-AUC 0.029, ROC-AUC 0.433. Real card fraud needs card-trained models; the transfer model is not a card model. Trained only on the synthetic card sets, the model reaches ROC-AUC 0.514 on the real test part, against 0.738 for the real-trained model: synthetic card fraud does not stand in for real card fraud. The unseen-payer subset has only 2 fraud cases, too few for any conclusion.

DELIVERABLE 2, DATASET EXPANSION EXECUTED

Cross-generator transfer: AMLworld to AMLgentex

The PDF ranks a second generator as the most informative next dataset. AMLgentex Sweden 100K was downloaded from Hugging Face, mapped to the AMLworld schema (mapping stated in `research/amlgentex_transfer.py`) and streamed through the same Rust feature code. Test = last 20% of the 112 daily steps. Prevalence differs by more than 10x between the sets, so compare ROC-AUC across rows and read PR-AUC against each row's prevalence.

TRAIN → TEST	PREVALENCE	ROC-AUC [95% CI]	PR-AUC [95% CI]
Sweden difficult: served HI model, zero-shot	0.0172	0.520 [0.514, 0.525]	0.018 [0.017, 0.018]
Sweden difficult: zero-shot, transfers only	0.0170	0.524	0.018
Sweden difficult: zero-shot, transfers sent as Wire	0.0172	0.467	0.016
Sweden difficult: XGBoost retrained in-domain (3 seeds)	0.0172	0.716	0.119
Sweden easy: served HI model, zero-shot	0.2968	0.532 [0.530, 0.534]	0.310 [0.308, 0.312]
Sweden easy: zero-shot, transfers only	0.2954	0.536	0.310
Sweden easy: zero-shot, transfers sent as Wire	0.2968	0.454	0.276
Sweden easy: XGBoost retrained in-domain (3 seeds)	0.2968	0.763	0.612
Sweden easy → difficult	0.0172	0.745	0.143
AMLgentex difficult → HI-Small slice	0.0016	0.153 [0.128, 0.181]	0.001 [0.001, 0.001]
Served HI model → HI-Small slice (reference)	0.0016	0.979 [0.969, 0.987]	0.408 [0.292, 0.532]

SAR rate by transaction type (difficult): CASH 100.0%, TRANSFER 1.7%. AMLgentex has its own type shortcut: every cash transaction is SAR, mirroring ACH in AMLworld. The reversed ranking on HI-Small follows from these two shortcuts meeting: AMLgentex teaches that cash is laundering and transfers (mapped to ACH) are normal, while HI-Small laundering is almost all ACH. Training on the easy set and testing on the difficult one (0.745) beats training on difficult itself, so more labels from a related generator help more than matching its difficulty. Strongest features retrained in-domain: `fmt_code`, `d_uniq_src`, `new_pair`, `s_fanout_24h`, `s_uniq_dst`.

SECOND GENERATOR, FULL FILE

SAML-D: the format-free model transfers, the served one does not

SAML-D (Oztas et al. 2023, UK typologies) has 9,504,852 payments. It was mapped to the AMLworld schema and streamed through the same Rust feature code. The test part is the last 20% in time (1,900,971 payments, 2,143 laundering).

MODEL	TEST ROC-AUC	TEST PR-AUC	RECALL AT 0.5% ALERTS
Served cascade, zero-shot	0.565	0.004	5.4%
HI-Small retrained, 31 features, zero-shot	0.664	0.003	4.6%
HI-Small retrained without format, zero-shot	0.802	0.020	13.2%
Trained on SAML-D (31 features)	1.000	0.991	100.0%
Trained on SAML-D without format	1.000	0.981	100.0%

Dropping the format feature raises zero-shot ROC-AUC from 0.664 to 0.802: the format shortcut learnt on AMLworld does not hold elsewhere (laundering rate by format here: ACH 0.06%, Cash 0.52%, Cheque 0.05%, Credit Card 0.06%, Wire 0.28%). The in-domain scores near 1.0 are not a real result. 47% of test laundering touches an account already seen laundering in train, against 9% of genuine payments, so the model can learn account identity. The unified pipeline below removes those payments and tests again.

UNIFIED TRAINING, EXECUTED

One format-free model over bank AML, cards and wallets

Every transaction-shaped dataset was streamed through the served Rust feature code with 13 new causal indicators (bursts, mule collection, same-pair structuring, amount and hour anomaly, payee age, new-payee velocity; `crates/rusetect/examples/research_dump.rs`, with a test that they read no future

payment). Each dataset is split by time 60/20/20; payment format and currency codes are left out, and so are label-adjacent columns (PaySim balances and flag). Each dataset gets the same total training weight. Leave-one-out is the model trained on all other datasets: the honest test of a new institution.

DATASET	PAYMENTS	TEST POSITIVES	SERVED ROC	POOLED, 29 BASE ROC	POOLED + 13 NEW ROC	LEAVE-ONE-OUT ROC	POOLED + NEW PR-AUC
hismall	5,078,345	1797	0.983	0.991	0.991	0.991	0.737
himed	31,898,238	10642	0.961	0.989	0.990	0.990	0.750
hi_large	179,702,229	71304	0.952	0.944	0.897	0.882	0.591
li_small	6,924,049	925	0.943	0.977	0.976	0.976	0.376
li_medium	31,251,483	3783	0.900	0.975	0.977	0.976	0.415
li_large	176,066,557	25149	0.875	0.861	0.751	0.863	0.261
samld	9,504,852	2143	0.564	1.000	1.000	0.803	0.959
amlgentex	7,648,937	26285	0.520	0.677	0.643	0.497	0.044
banksim	594,643	1280	0.934	0.996	0.997	0.940	0.854
sparkov	1,852,394	1349	0.469	0.989	0.989	0.751	0.798
paysim	6,362,620	4254	0.360	0.914	0.917	0.349	0.314
tabformer	24,386,900	4833	0.635	0.861	0.883	0.458	0.067
ieee	590,540	4064	0.433	0.598	0.621	0.435	0.060

Rule fixed before test was read: adopt the new indicators if the mean ROC-AUC gain across datasets has a paired bootstrap 95% interval above zero. Gain -0.0108 [-0.0114, -0.0102]. **Not adopted.** The new indicators raise ROC-AUC on hismall, himed, li_medium, banksim, sparkov, paysim, tabformer, ieee and lower it on hi_large, li_small, li_large, samld, amlgentex. Some rank among the strongest features (`d_uniq_src`, `d_fanin_1h`, `d_out_n`, `pair_usd_24h`, `new_bank_pair`, `s_usd_1h`, `hour`,

`s_out_mean_usd`), but they mostly re-express what the 29 base features already carry, so they stay research-only.

Pooling works in-domain: the pooled format-free model has higher ROC-AUC than the served cascade on 11 of 13 datasets (and higher PR-AUC on all). Generalisation does not: trained without a dataset, the model stays below 0.75 ROC-AUC on amlgentex, paysim, tabformer, ieee. Each generator and fraud type needs some of its own labels, so a new institution must label data before it trusts the score.

Unseen accounts

DATASET	TEST POSITIVES KEPT	SERVED ROC	POOLED + NEW ROC	POOLED + NEW PR-AUC
hismall	1374	0.996	0.997	0.778
samld	1141	0.583	1.000	0.956
amlgentex	0	none left	none left	none left
banksim	0	none left	none left	none left
sparkov	15	0.570	0.903	0.601
paysim	4244	0.360	0.917	0.315
ieee	2	0.807	0.852	0.012

Test payments touching any account seen in a positive training payment were removed. This is the check against learning account identity (SAML-D above). No positives remain in amlgentex, banksim: every test fraud there involves an account or merchant already seen in training fraud, so these sets cannot test unseen accounts. SAML-D stays at ROC-AUC 1.000 on unseen accounts, so account reuse does not explain its easy separation: the generator itself makes laundering easy to tell apart, and SAML-D scores should not be read as real-world performance.

Structuring attack, HI-Small demo window

MODEL	CLEAN RECALL	SPLIT IN 4, SAME MINUTE	SPLIT IN 4, 30 MIN APART
served_cascade	37%	2%	2%
pooled_base	15%	19%	3%
pooled_base_new	13%	18%	1%

97 laundering payments in the window; the served models' block threshold is replaced by an equal alert rate per model. A laundering payment counts as caught if any part is flagged. Spreading the parts 30 minutes apart defeats every model (served_cascade 2%, pooled_base 3%, pooled_base_new 1%), including the one with same-pair structuring indicators. Each part is scored alone, so the fix is to sum evidence per account over the window and alert on the account, not to add more per-payment features.

OTHER FRAUD TYPES, EXECUTED

Crypto, credit, cards, logins and graphs

Datasets whose schema does not fit a payment stream were tested on their own, with a time split wherever the data has time, and XGBoost as the common learner. An arrow shows the gain from adding graph or neighbour aggregates, the architecture question the audit raises. These are capability checks, not served models.

DATASET	FRAUD TYPE	ROWS / POSITIVES	TEST PR-AUC	TEST ROC-AUC	NOTES
Elliptic (Bitcoin, real labels)	Crypto	46,564 / 4,545	0.785 → 0.804	0.925 → 0.938	After time step 43 (dark market shutdown) PR-AUC falls to 0.048
Elliptic++ wallets	Crypto	367,472 / 28,601	0.548	0.917	rows are wallet x time step; a wallet can appear in train and test steps, as in the dataset's own benchmark

DATASET	FRAUD TYPE	ROWS / POSITIVES	TEST PR-AUC	TEST ROC-AUC	NOTES
DGraph-Fin (loan default fraud graph)	Credit	3,700,550 nodes	0.026 → 0.038	0.713 → 0.784	official validation split (test labels hidden in the mirror); paper reports test ROC-AUC about 0.72 (MLP) to about 0.78 (best GNNs)
ULB credit card	Card	284,807 / 492	0.796 [0.708, 0.883]	0.979	chronological 60/20/20 by Time
IEEE-CIS (Vesta)	Card not present	590,540 / 20,663	0.493 → 0.499	0.891 → 0.893	chronological 60/20/20 by TransactionDT; device and identity columns added
RBA logins (account takeover)	Login	31,269,264 / 141	0.035	0.961	29 takeovers in test; 'Is Attack IP' is excluded: it is a label-adjacent flag, not something a bank sees before the login
Ethereum addresses (real flags)	Crypto	9,816 / 2,179	0.996	0.999	Random 5-fold split (no time). Token fields are empty for 38% of flagged and 0% of genuine addresses, a collection artefact the model can use, so near-perfect scores are not evidence of real-world accuracy
BitcoinHeist (real ransomware addresses)	Crypto	2,916,697 / 41,413	0.002	0.603	Split by date; 988 ransomware in test; at the validation F1 threshold precision 0.2%, recall 8.4%, FPR 8.826%
Berka (real Czech bank)	Credit default	682 loans / 76 bad	0.799	0.969	12 bad loans in test; credit default, not fraud: the audit's 'behaviour realism' use (class D/E); few positives, so the interval is wide
YelpChi (review graph)	Not payments	45,954 / 6,677	0.814 → 0.848	0.943 → 0.955	review spam, not payments; architecture comparison only, as the audit classes it (D/E)
InstrumentReviews (review graph)	Not payments	11,944 / 821	0.888 → 0.908	0.977 → 0.982	review spam, not payments; architecture comparison only, as the audit classes it (D/E)

Neighbour aggregates help on every graph (Elliptic, DGraph and both review graphs), which supports adding graph features before a graph neural network. Elliptic shows the limit of any static model: after the dark market shutdown the illicit pattern changes and recall collapses, so drift monitoring must trigger retraining.

SCAM MESSAGE SCREEN, EXTERNAL TEST

Messages the screen has never seen

Source: bolewara/hinglish-scam-text-dataset (CC BY 4.0), rows not in UCI SMS Spam. Only messages absent from the training corpus were kept.

SET	MESSAGES	SCAM	RECALL, CAUTION OR ABOVE	RECALL, HIGH OR ABOVE	FALSE ALARMS
All new messages	87	47	85.1%	68.1%	0
Hinglish subset	30	13	84.6%	76.9%	0

Missed Hinglish scams are a recharge offer and an instant loan offer: types the rule set does not cover yet.

DELIVERABLE 4

Security and threat assessment

Status says how strong the evidence is. "Experimentally demonstrated" means measured above; "verified in code" means read in the current source; "theoretical" means plausible but untested. Framework references are to NIST AI 100-2 E2025, the OWASP API Security Top 10 (2023) and MITRE ATLAS.

THREAT	EVIDENCE	IMPACT	STATUS IN THIS BRANCH	REFERENCE
Evasion: payer switches payment format	Experimentally demonstrated	Block recall 37% to 0%	Fixed: hardened risk adds a format-free second opinion; test PR-AUC under the attack 0.687 (was 0.004), clean 0.720	NIST AI 100-2 evasion; MITRE ATLAS AML.T0015
Evasion: structuring into smaller payments	Experimentally demonstrated	Block recall to 1%	Partly fixed: 24 h pair aggregation before calibration; four-part split block recall 26% (was 5%); clean recall unchanged	NIST AI 100-2 evasion

THREAT	EVIDENCE	IMPACT	STATUS IN THIS BRANCH	REFERENCE
Missing or unknown format and currency silently imputed	Experimentally demonstrated	7,037 genuine payments escalated in 3 h	Fixed: 422 on missing or unknown values; test added	OWASP API8 misconfiguration
Rate limit keyed on client-supplied X-Forwarded-For	Verified in code	Unlimited requests; unbounded memory per spoofed client	Fixed: header trusted only on Vercel or with RUSETECT_TRUST_PROXY=1; idle clients evicted	OWASP API4 unrestricted resource consumption
Alerts, evidence packs, STR drafts, cases and audit readable without a key	Verified in code	Disclosure of suspicion (tipping off) and account identifiers	Fixed when keys are configured; demo mode stays open by design	OWASP API1/API2
Demo keys enabled whenever RUSETECT_ANALYST_KEYS is unset	Verified in code	Fail-open: a misconfigured deployment accepts published keys	Fixed: the server refuses to start without RUSETECT_ANALYST_KEYS or an explicit RUSETECT_DEMO=1 (test added)	OWASP API8
Hard-coded fallback secrets (consortium HMAC, entity hash salt)	Verified in code	Account tokens can be reversed by enumeration when defaults are used	Fixed: with analyst keys set the server refuses to start without the consortium secret, hash salt and signing key (test added)	NIST AI 100-2 privacy
Ephemeral signing key and in-memory nonce cache per serverless instance	Verified in design	Signatures from one instance cannot be checked after restart; nonces can be replayed on another instance	Partly fixed: a signing key is required outside demo mode; nonces still per instance (move to Redis)	Integrity
Unbounded account state from API traffic	Verified in code	Memory exhaustion with random account ids	Fixed: a cap on accounts (RUSETECT_MAX_ACCOUNTS, default 2,000,000); past it, payments that would add accounts get 503 and known accounts still score	OWASP API4
Label poisoning through analyst feedback	Theoretical	Released or blocked verdicts would feed retraining	Plan: two-person rule on labels used for training; audit trail already signed	NIST AI 100-2 poisoning
Model extraction through scores and reasons	Theoretical	API returns risk, both halves and SHAP reasons	Plan: return bands only to payment channels; full reasons to analysts	NIST AI 100-2 privacy
Dependency vulnerabilities	Checked	cargo audit: 0 advisories in 199 crates (RustSec, 1,453 advisories)	Keep cargo audit in CI	Supply chain

DELIVERABLE 3

Literature, verified

Every identifier below was resolved against arXiv or Semantic Scholar during this review (raw responses in `research/out/s2_refs.json`). All arXiv identifiers cited by the PDF resolved to the papers it names. Recent AMLworld work found during the search (BlazingAML, arXiv 2604.12241; TRACK, arXiv 2611.06035; federated cross-bank context, arXiv 2610.35996; AMLworld-Compact, arXiv 2610.13627) reports results under transductive or account-window protocols, so none is directly comparable with RuseTect's strictly causal payment-level protocol.

WORK	IDENTIFIER	WHAT IT CONTRIBUTES	RELEVANCE HERE
Altman et al., NeurIPS 2023 D&B	arXiv 2306.16424	AMLworld generator and HI/LI sets	Source of all payment results; its typology mix explains the ACH shortcut
Östman et al., 2025 (AMLgentex)	arXiv 2506.13989	Configurable generator, 12 banks, label noise	Second generator used in the transfer test above
Oztas et al., IEEE ICEBE 2023 (SAML-D)	doi 10.1109/ICEBE59045.2023.00028	28 typologies, 9.5M payments	Third generator for transfer (Kaggle only)
Blanuša et al., ICAIF 2024 (Graph Feature Preprocessor)	arXiv 2402.08593	Past-only subgraph pattern features for boosted trees	Best supported graph upgrade; addresses structuring via fan and cycle counts
Egressy et al., AAAI 2024 (Multi-PNA)	arXiv 2306.11586	GNNs for directed multigraphs on AMLworld	Graph baseline with a non-causal window
Lin et al., ICAIF 2024 (FraudGT)	doi 10.1145/3677052.3698648	Graph transformer, minority F1 on AMLworld	Comparison point; not strictly causal
Maganti, 2026	arXiv 2604.19514	Strict inductive re-evaluation of GNNs on Elliptic	Supports the repo's causal protocol; trees beat GNNs
Kim et al., 2024	arXiv 2404.00060	TGN for financial graph anomalies (DGraph)	Temporal GNN research arm
Rossi et al., 2020 (TGN)	arXiv 2006.10637	Temporal graph networks	Method behind the research arm

WORK	IDENTIFIER	WHAT IT CONTRIBUTES	RELEVANCE HERE
Poursafaei et al., NeurIPS 2022	arXiv 2207.10128	EdgeBank: memorisation baseline for dynamic graphs	Any temporal GNN result needs this baseline
Jing et al., 2024 (CFDB)	arXiv 2404.14746	Customer-level benchmark from SAML-D and AMLworld	Template for account (mule) labels
Nahimana et al., 2026	arXiv 2608.15447	Mobile money laundering detection (Rwanda) on synthetic data	Confirms the lack of real mobile money labels
Braithwaite et al., 2025 (nuFormer)	arXiv 2507.23267	Transaction transformer with joint fusion	Small gains over trees at very large scale
Grinsztajn et al., NeurIPS 2022	arXiv 2207.08815	Trees beat deep nets on tabular data	Supports keeping boosted trees
Geirhos et al., Nature MI 2020	doi 10.1038/s42256-020-00257-z	Shortcut learning	Frames the payment format finding
Kireev et al., NDSS 2023	arXiv 2208.13058	Cost-aware adversarial robustness for tabular fraud models	Method for realistic, costed evasion tests
Cartella et al., SafeAI 2021; Ballet et al., 2019	arXiv 2101.08030; 1911.03274	Adversarial attacks on tabular and fraud data	Feature-space attack baselines
Lunghi et al., 2023	arXiv 2307.01390	Adversarial learning in real-world fraud detection	Threat model for payment-side attackers
Niculescu-Mizil and Caruana, ICML 2005	doi 10.1145/1102351.1102430	Calibration of boosted models	Basis for the calibration fix
Dal Pozzolo et al., IEEE SSCI 2015	doi 10.1109/SSCI.2015.33	Calibration after undersampling	Explains the 12x score inflation
van den Goorbergh et al., JAMIA 2022	doi 10.1093/jamia/ocac093	Harm of imbalance corrections	Argues against SMOTE-style resampling
Saito and Rehmsmeier, PLoS ONE 2015	doi 10.1371/journal.pone.0118432	PR over ROC under imbalance	Metric choice
Correa Bahnsen et al., ESWA 2015	doi 10.1016/j.eswa.2015.04.042	Example-dependent cost-sensitive learning	Cost model with amounts
Lebichot et al., IJDSA 2021	doi 10.1007/s41060-021-00258-0	Incremental learning with delayed labels	Retraining cadence

WORK	IDENTIFIER	WHAT IT CONTRIBUTES	RELEVANCE HERE
Alves et al., TMLR 2024	arXiv 2403.06906	Learning to defer under analyst capacity	Capacity-aware review band
Han et al., NeurIPS 2022 (ADBench)	arXiv 2206.09426	Anomaly detection benchmark	Why unsupervised scorers stay monitors
Lundberg et al., Nature MI 2020	doi 10.1038/s42256-019-0138-9	TreeSHAP	Reason codes
Jesus et al., FAccT 2021; Weerts et al., 2019	doi 10.1145/3442188.3445941; arXiv 1907.03324	SHAP does not reliably help analysts	Limits on explanation claims
Jesus et al., NeurIPS 2022 (BAF)	arXiv 2211.13358	Bank Account Fraud datasets	L0 onboarding data
Bellei et al., 2024 (Elliptic2); Elmougy and Liu, KDD 2023 (Elliptic++)	arXiv 2404.19109; doi 10.1145/3580305.3599803	Real, anonymised Bitcoin labels	Real-label sanity check for graph features
Huang et al., NeurIPS 2022 (DGraph)	arXiv 2207.03579	Real financial graph with missing features	Robustness of graph methods
NIST AI 100-2 E2025	csrc.nist.gov/pubs/ai/100/2/e2025/final	Adversarial ML taxonomy	Threat model vocabulary

DELIVERABLE 5

Research gap and novelty

A new detector architecture would not be novel: boosted trees with engineered graph features are the documented state of practice on AMLworld, and RuseTect's own GNN and sequence results agree. The defensible contribution is methodological, and this review strengthens it with three results the literature found in this search does not report together:

- **Shortcut audit of a public AML benchmark.** A single categorical field dominates both the label distribution and the served model, and a one-step payer-side change defeats it. Published AMLworld results do not report format-ablated or format-switched recall.

- **Benchmark composition.** The trailing laundering tail inflates any metric on the last days of HI-Small; a corrected protocol is a short, citable contribution.
- **Serving-parity evaluation.** Every number here comes through the production feature code (Python and Rust agree to 3e-8), which turns robustness tests into tests of the deployed system rather than of an offline copy.

Together with the cross-generator result, this supports a workshop paper on *what survives a causal, serving-parity, adversarial evaluation of AML detection on synthetic data*. The claim must stay within synthetic data: no public labelled UPI data exists (confirmed in the PDF and not contradicted by this search).

DELIVERABLE 6

Methodology improvements, ranked by evidence

CHANGE	WHY (EVIDENCE)	DATA	METHOD	METRIC
Drop the LightGBM half; serve calibrated XGBoost	No measurable PR-AUC gain; halves latency and makes reasons exact	Existing models	Isotonic on validation; refit bands under a pre-registered rule	PR-AUC unchanged within CI; ECE ↓
Format-robust model	Removes the dominant shortcut and the one-step evasion	HI/LI full	Retrain without fmt_code (or monotone cap); compare under format-switch attack	Recall under attack; clean PR-AUC cost (PDF: 0.728 → 0.701)
Structuring-aware features	Splitting removes the block band	HI/LI full, AMLgentex	Past-only sums and counts per sender and payee over 1 h and 24 h; GFP fan and cycle counts	Recall under split attack; per-typology recall
Account-level mule score	Banks freeze accounts, not payments	AMLworld patterns, AMLgentex (account SAR), SAML-D	CFDB-style aggregation; time to first detection	Account PR-AUC; days to detection
Cost and capacity-aware thresholds	Bands are optimal for one implicit cost ratio	Existing	Choose thresholds by expected cost and daily alert budget; learning to defer later	Cost at budget; precision@k

CHANGE	WHY (EVIDENCE)	DATA	METHOD	METRIC
Unsupervised anomaly scoring as a monitor only	Repo evidence: IF recovers 0 of 4 held-out typologies	Existing	Keep for drift alarms, not decisions	Alarm delay after injected drift

Not recommended: replacing trees with a GNN (strict inductive evidence against), SMOTE-style resampling (miscalibration without AUC gain), and a dedicated adversarial training module before the cheap feature fixes above are measured.

DELIVERABLE 8

Independent jury

Six reviewer perspectives, each written against the measured evidence above rather than the project's own framing.

Senior academic researcher

`Accept for a workshop after revision`

- The contribution is evaluation methodology; the paper must say so in the title and abstract.
- 0.711 is reported on a test window whose tail is a simulation artefact. Re-report on September 9 to 10 and with the tail excluded.
- All evidence is synthetic. State this in every claim and keep generalisation claims to the measured transfer.

Machine learning researcher

`Major revision`

- The LightGBM average and the L1 gate are not justified on accuracy; ablate them with intervals.
- Scores are uncalibrated by about 12x after negative subsampling. Calibrate before thresholding.
- Report seeds and confidence intervals for every model comparison; 0.7107 versus 0.7108 is not a result.

Cybersecurity specialist

`Not deployable as is`

- A payer-controlled field dominates the score and a one-step change defeats the block band.
- Structuring to four parts removed the block band; pair aggregation now recovers part of it, account-level alerts are still needed.
- Fail-open defaults (demo keys, secrets, silent imputation) must fail closed outside demo mode. All three now do.

Dataset and statistics expert

`Concerns about validity`

- 97 positives in the slice and 1,797 in the full test are too few for three-decimal claims without intervals.
- Prevalence differs 100x between AMLworld and AMLgentex; compare ROC-AUC or prevalence-adjusted metrics across generators.
- Amount-weighted metrics are dominated by two payments; report them with and without extremes.

Industry research engineer

`Promising engineering, weak operations story`

- Serving parity and streaming state are strong and rare.
- Thresholds should come from an alert budget and a cost ratio, which only a bank can supply; ask for them.
- Serverless state breaks fusion, nonces and signing; the demo should say so on the page.

Critical reviewer

`Reject in current form, accept the reframed version`

- Several Round 1 claims (97% precision, calibrated bands, 15x fewer alerts) are contradicted by the repo's own files.
- "Detects unseen patterns" is not supported across generators.
- Novelty must rest on the audit, the shortcut and the protocol, not on the cascade.

Questions a demanding panel will ask, with the answer the evidence now gives

QUESTION	ANSWER FROM THIS REVIEW
What happens if a launderer pays by cheque instead of ACH?	Block recall falls from 37% to 0% (served model, slice).
Is 0.711 inflated by the end of the dataset?	Probably: about 36% of test positives sit in a 1,108 payment tail at 59% prevalence. Re-measure.

QUESTION	ANSWER FROM THIS REVIEW
Does the LightGBM half help?	No: 0.007 [-0.039, 0.046] for XGBoost alone minus served.
Are the scores probabilities?	No: mean score 0.0174 against observed 0.0015.
Do the features work on another generator?	No. The served model is at chance on AMLgentex (0.520); retrained, the same features reach 0.716; trained on AMLgentex and tested on HI-Small, the ranking reverses (0.153).
Can splitting payments avoid a block?	Yes: four parts leave 1% blocked.

DELIVERABLES 7 AND 9

Prioritised plan

Ranked by scientific value per unit of effort, with data that exists today. Priority 1 turns every existing number into a defensible one and should finish before new modelling.

Priority 1: essential corrections

ITEM	HOW TO DO IT	DATA	EFFORT
Re-measure the headline without the test tail	Score HI-Small test as Sep 8 part + Sep 9 to 10 + Sep 11 to 18 separately; report AP with bootstrap CI for each	HI-Small full	0.5 week
Format robustness	Retrain L2 without fmt_code (keep everything else); evaluate clean and under format switch with research/analyze_slice.py section G on the full test	HI, LI full	1 week
Single calibrated L2	Serve XGBoost alone with isotonic calibration fitted on validation; refit bands under a rule written before reading test; update cascade_config.json only if it passes	HI, LI validation	1 week
Intervals everywhere	Wrap every headline in the paired bootstrap used here; add seeds for retrained models	Existing	0.5 week
Fail-closed configuration	Require RUSETECT_DEMO=1 for demo keys; refuse to start without consortium secret, hash salt and signing key otherwise	None	0.5 week

ITEM	HOW TO DO IT	DATA	EFFORT
Rewrite the claims	Bring the Round 1 paper and slides in line with the measured numbers (PDF claim table plus findings 1 to 6)	None	1 week

Priority 2: high-value experiments

ITEM	HOW TO DO IT	DATA	EFFORT
Structuring defence	Add past-only 1 h and 24 h sums per sender and per payee and GFP-style fan and cycle counts; measure recall under split attacks	HI, LI, AMLgentex	2 to 3 weeks
Cross-generator matrix	Extend research/amlgentex_transfer.py to SAML-D and AMLworld Medium; report ROC-AUC and prevalence-adjusted AP	SAML-D, HI-Medium	2 weeks
Account-level mule detection	Aggregate payment scores per account (CFDB method); evaluate on AMLgentex account SARs	AMLgentex, AMLworld	2 weeks
Cost and capacity thresholds	Choose bands from expected cost and a daily alert budget; publish the cost ratios used	Existing	1 week

Priority 3: advanced directions

ITEM	HOW TO DO IT	DATA	EFFORT
Costed adversarial evaluation	Kireev et al. style attacks where each feature change has a cost for the launderer	HI full	3 weeks
Temporal GNN research arm	TGN on identical causal splits with EdgeBank and GFP+XGBoost baselines; adopt only if validation gain clears the rule	AMLworld S/M	3 to 4 weeks
Drift and label delay	Walk-forward on HI-Large and AMLgentex typology switch; measure detection delay and recovery	HI-Large, AMLgentex	2 weeks
Federated or partitioned banks	AMLgentex 12 banks: local vs pooled vs federated ring recall	AMLgentex	3 weeks

DELIVERABLE 10

What this review changed in the repository

- `crates/rusetect/examples/research_dump.rs` : streams any AMLworld-format CSV through the served engine and writes the model inputs and all three scores.
- `research/make_streams.py` , `research/analyze_slice.py` , `research/amlgentex_transfer.py` , `research/build_report.py` : reproducible experiments and this page.
- API: `payment_format` is required and validated; payment and receiving currencies are validated (tests added).
- Rate limiter no longer trusts a client-supplied `X-Forwarded-For` outside Vercel, and evicts idle clients.
- With configured analyst keys, alerts, evidence packs, STR drafts, cases and the audit log require a key, and the server refuses to start without its signing key, consortium secret and hash salt.
- Models and pages are embedded in the server binary, which fixes the Vercel deployment (every page returned 404 because Vercel runs the server binary without the repository files).

Reproduce

```
cargo build --release -p rusetect --example research_dump
python3 -I research/make_streams.py
python3 -I research/analyze_slice.py
python3 -I research/amlgentex_transfer.py <folder with sweden_difficult.csv and sweden_easy.csv>
python3 -I research/build_report.py
```

Limitations: the HI-Small results here come from an 8 hour slice with 97 evaluation positives and five hours of account history; they measure effects on the served models, not the full-protocol level. The test-tail figures come from an independent audit of the same file and have not been re-counted here.

AMLgentex mapping choices (daily timestamps, format and currency) are stated in the script and are part of the measured gap.